Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 6 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

1. Week 6 Overview

Week 6 focused on completing the essential Minimum Viable Product (MVP) functions of the Automated Grocery Inventory Dashboard and demonstrating Git collaboration and release management.

The major objectives were:

- Complete the inventory item catalogue.
- Implement stock-in and stock-out transactions.
- Maintain inventory quantities automatically.
- Display stock status based on quantity and reorder level.
- Provide search and category filtering.
- Implement exception alerts for low-stock and out-of-stock items.
- Demonstrate feature branching and Git collaboration.
- Create and resolve a merge conflict.
- Create a release-ready baseline and Git tag.
- Update the project backlog.

1.1 Week 6 MVP Status

Requirement	**Status**
Item catalogue	Completed
Create/update inventory items	Completed
Stock In / Stock Out	Completed
Transaction history	Completed
Stock status	Completed
Search and category filtering	Completed
Exception alerts	Completed
Feature branch workflow	Completed
Merge conflict creation and resolution	Completed
Release baseline	Completed
Version tag `v0.6.0`	Completed
Updated backlog	Completed

Table 1: Week 6 Completion Status

2. Functional MVP Implementation

2.1 Item Catalogue

The item catalogue provides a centralized interface for managing grocery inventory items.

The implemented functionality includes:

- Add new inventory items.

- Edit existing items.
- Delete items.
- Store item name, category, quantity, unit, price and reorder level.
- Display the current inventory status.
- Search items using item name or category.
- Filter items by category.

The catalogue also displays summary information such as total items, items in stock, low-stock items and out-of-stock items.

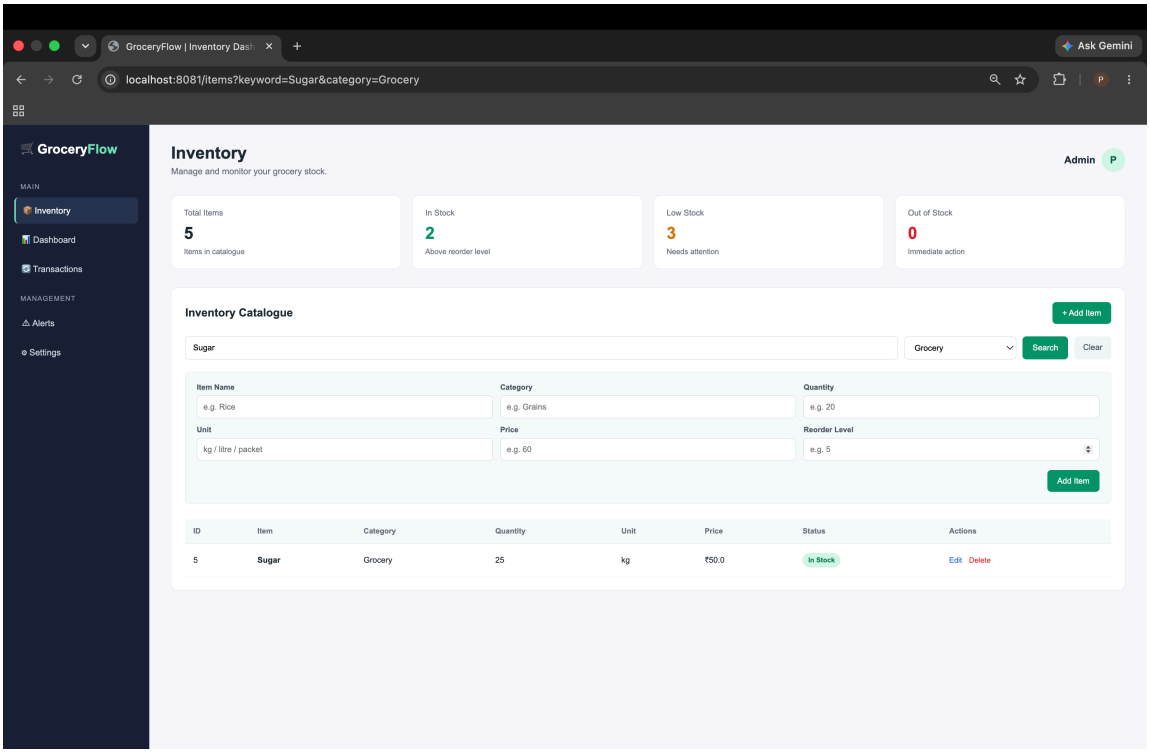


Figure 1: Inventory Catalogue with Search, Filtering and Stock Status

2.2 Stock Transactions

The transaction module was implemented to update inventory quantities through stock movements.

Two operations are supported:

- **Stock In:** increases the quantity of the selected item.
- **Stock Out:** decreases the quantity of the selected item.

Each transaction records the item, transaction type, quantity, date/time and optional note.

This creates a transaction history that allows stock movements to be tracked.

2.3 Stock Status

The inventory status is calculated using the current quantity and reorder level.

- **In Stock:** quantity is above the reorder level.
- **Low Stock:** quantity is greater than zero but at or below the reorder level.
- **Out of Stock:** quantity is zero.

This status is reflected in the inventory catalogue and alert module.

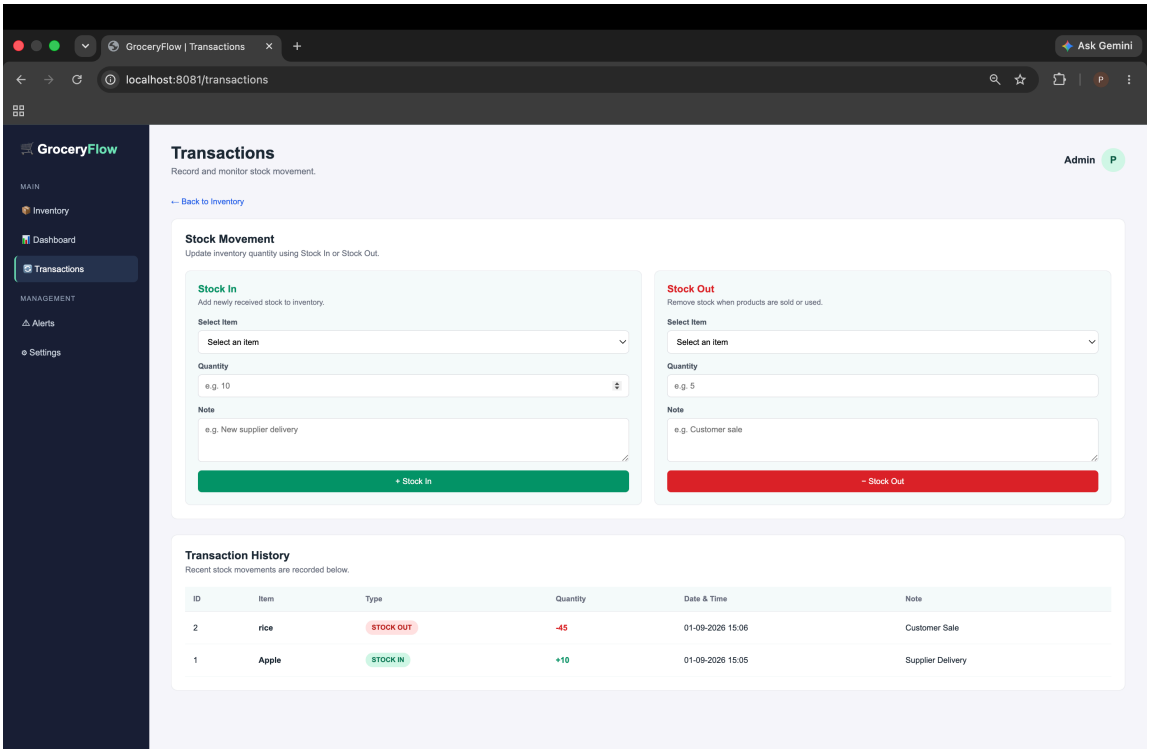


Figure 2: Stock In, Stock Out and Transaction History

3. Exception Alerts

The exception alert module identifies inventory items that require attention.

Two important conditions are handled:

1. Low-stock items.
2. Out-of-stock items.

The alerts page provides a separate view of items requiring stock attention. It displays the current quantity and reorder level so that the user can immediately identify why an item has been flagged.

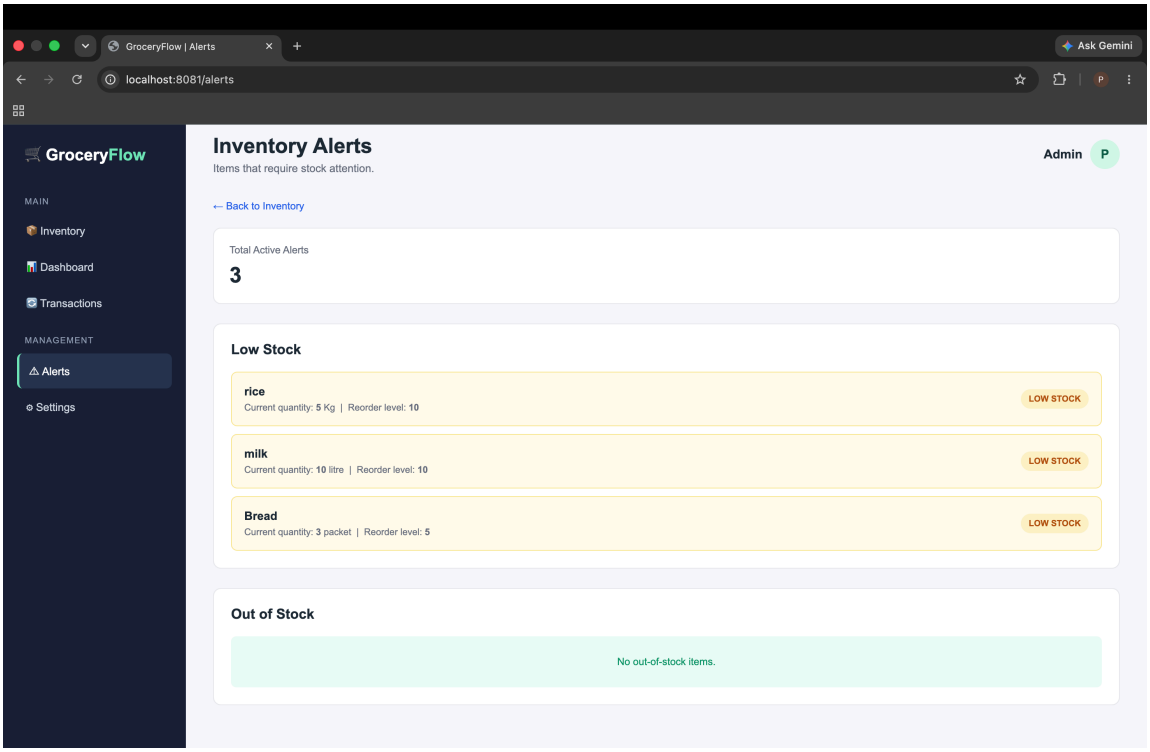


Figure 3: Inventory Exception Alerts

The alert functionality completes the essential monitoring part of the MVP by connecting inventory quantities with actionable stock information.

4. Git Collaboration and Release Management

Week 6 also focused on demonstrating a structured Git workflow.

The primary development branch used for the transaction work was:

`feature/inventory-transactions`

The item catalogue work was already integrated through the `feature/item-catalogue` branch.

4.1 Feature Development

The transaction functionality and exception alert implementation were committed to the transaction feature branch.

Important commits included:

```
d17f354  Implement inventory stock transactions
5eee3e6  Add inventory exception alerts
b850cd6  Add Week 6 release baseline
```

The feature branch was then merged into `develop`.

4.2 Merge Conflict Demonstration

To demonstrate conflict handling, the file:

`week6-conflict.txt`

was intentionally modified on both branches.

A merge was then performed:

```
git merge feature/inventory-transactions
```

Git reported an `add/add` conflict. The conflict was manually resolved and the resolved file was staged.

The merge was completed with:

```
git add week6-conflict.txt
git commit -m "Resolve Week 6 merge conflict"
```

The resulting commit was:

`acea9b0 -- Resolve Week 6 merge conflict`



Figure 4: Git Branching, Merge Conflict Resolution and Release Tag

5. Release Baseline and Updated Backlog

After resolving the merge conflict, the Week 6 release baseline was completed on the `develop` branch.

The release was tagged as:

v0.6.0

The tag was pushed to GitHub using:

```
git tag -a v0.6.0 -m "Week 6 MVP release"
git push origin v0.6.0
```

The final branch state was verified using:

```
git status
```

The result was:

```
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
```

6. Conclusion

Week 6 completed the essential MVP functionality of the Automated Grocery Inventory Dashboard.

The system can now manage grocery items, perform stock-in and stock-out operations, maintain inventory quantities, classify stock status, search and filter catalogue items, and identify low-stock and out-of-stock exceptions.

The DevOps objectives were also completed through feature branching, GitHub collaboration, intentional conflict creation, conflict resolution, release baseline creation and version tagging.

The Week 6 release is represented by the Git tag:

`v0.6.0`

The final `develop` branch was verified to be clean and synchronized with the remote repository. Therefore, the Week 6 MVP and its associated Git collaboration and release-management requirements are complete.